

Stack (All-Questions)

Question # 01

```
class Stack
```

```
{  
    private:
```

```
        int* ary;
```

```
        int top;
```

```
        int cap;
```

```
// data member
```

```
    public:
```

```
        Stack(int s);
```

```
// constructor
```

```
{
```

```
    cap = s;
```

```
    ary = new int[cap];
```

```
    top = -1;
```

```
}
```

```
// initialize
```

```
    ~Stack()
```

```
// destructor
```

```
{
```

```
    delete[] ary;
```

```
}
```

```
    void push(int x)
```

```
// insert element
```

```
{
```

```
    if (top == cap - 1)
```

```
// if cap is full
```

```
{
```

```
        return;
```

```
}
```

```
    top++;
```

```
    ary[top] = x;
```

```
// increment top
```

```
// assign top index new
```

```
}
```

```
void pop()
```

```
// delete element
```

```
{  
    if (top == -1)
```

```
// if no element
```

```
    return;
```

```
    prompt element; top--;
```

```
// delete last entered element
```

```
}
```

```
void peek()
```

```
// see last enter elem
```

```
{  
    if (top == -1)
```

```
// if no element
```

```
    return;
```

```
    prompt element;
```

```
// cout element
```

```
}
```

```
bool is_empty()
```

```
// checking emptiness.
```

```
{
```

```
    return top == -1;
```

```
// return true if top == -1
```

```
}
```

```
void display()
```

```
// display all elements
```

```
{
```

```
    if (top == -1)
```

```
// if no element
```

```
        return;
```

```
    }
```

```
    for (int i = top; i >= 0; i--)
```

```
// run from last to 1st
```

```
    {
```

```
        prompt elements;
```

```
// cout elements
```

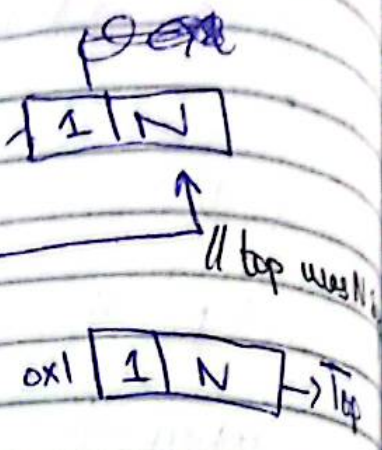
```
    }
```


Question # 02

is explained in basic operations of Stack
(DRY-RUN)

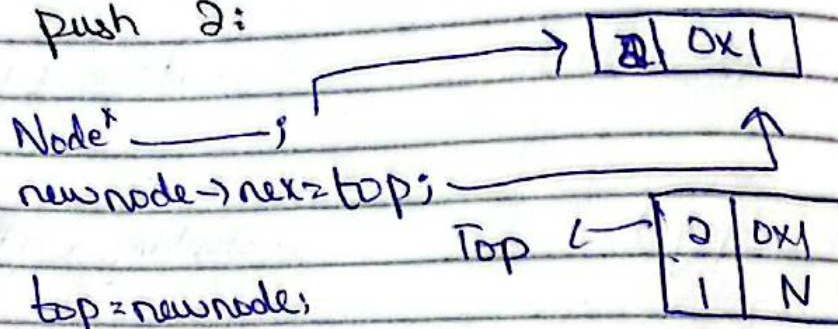
1) Suppose: user push 1:

Node* newnode = new Node(1);
newnode->next = top;
top = newnode;

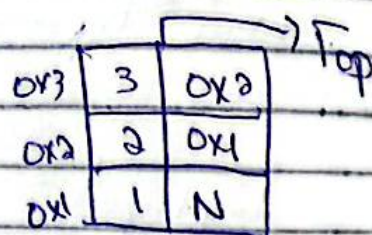


2) user push 2:

Node* newnode = new Node(2);
newnode->next = top;
top = newnode;



3) user push 3:



4) Now: peek

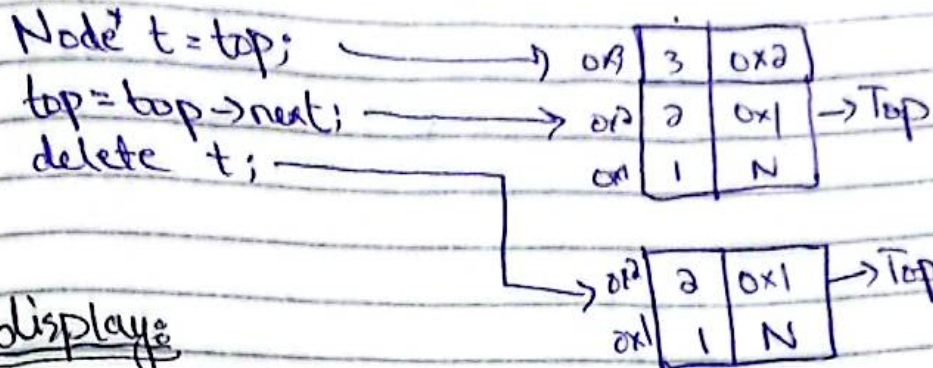
if (top == NULL) { } top = 0x3

cout << top->data;

3

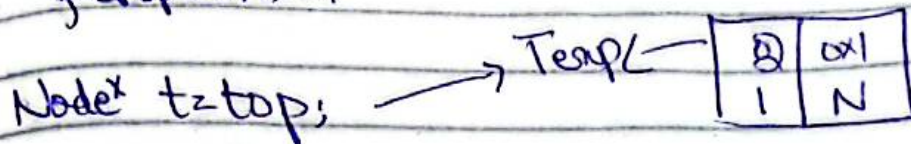
New Pop:

if (top == Null) X top = 0x3



New display:

if (top == N) X



1) while (t != N) ✓

{
cout << t->data;
t = t->next;
}

2
0x1

1) while (t != N) ✓

{
cout << t->data;
t = t->next;
}

1

t = N

1) while (t != N) X t = N

Question #03

// Remaining part is same


```

void insertatbottom(int n)
{
    if (isEmpty()) { return; }
    int t = pop();
    insertatbottom(n);
    push(t);
}

```

// if no element

// pop into t
// recursing
// push temp

```

void reverse()
{

```

// reverse func

```

    if (isEmpty()) { return; }

```

// if no element

```

    int t = pop();
    reverse();
    insertatbottom(t);
}

```

// pop into t
// recursing
// call func

(DRY_RUN)

Suppose: our stack is :

3	2	1	0
2	0x1	-> top	
1	N		

reverse: (a)

if (—) X

```

int t = pop();    t = 2
reverse();

```

reverse: (b)

if (—) X

```

int t = pop();    t = 1
reverse();

```

reverse: (c)

if (empty) ✓
return

insertatbtm(temp): (1)

insertatbtm(list a): (b)

if (empty) ✗

int t = pop()

t = []

insertatbtm

insertatbtm(list a): (e)

is (empty) ✓

push(1);

return

0 | 1 | N

insertatbtm(temp); t = 2

insertatbtm(list v): (a.f)

if (empty) ✗

int t = pop() t = 1;

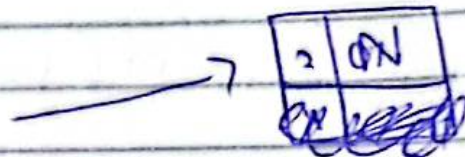
call insertatbtm;

insertatbtm;

is (empty)

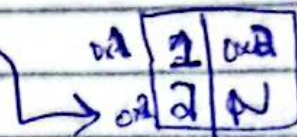
push(2);

return



push(1);

return;



Reverse List ↑

Question #04

```

void sortedinsert(int w)
{
    if (isep() || peak() < w)
    {
        push(w);
        return;
    }
    int t = pop();
    sortedinsert(w);
    push(t);
}

```

// if val > top or stack is empty
 // push val
 // return to caller

A store pop into t
 // recursive call
 // push t

```

void sortstack()
{
    if (isep())
    {
        return;
    }
    int t = pop();
    sortstack();
    sortedinsert(t);
}

```

// if empty
 // return to caller

// store pop into t
 // recursive call
 // call function

(DRY RUN)

Suppose our stack is:

0x1	2	0x2
0x2	2	0x3
0x3	3	N

a) sortstack:

```

if (isep())
{
    int t = pop();    t = 2
    sortstack();
}

```

sortstack(2):

if (-) x

int t = pop() t = 1

sortstack;

sortstack(3):

if (-) x

int t = pop() t = 3

sortstack;

stack [0]

sortstack(4):

if (-) ✓

return; (to sortstack3)

sortedingest(t): [t=3]

sorted(t):

if (t > 1) ✓

push(x);

return; (to stack2)

[3 | N]

sorted(t)

[t=2]

sorted(t):

if (-) ✓

push(x);

return (to stack1)

1	0x3
3	N

sorted(t)

t=2:

sorted(t)

if (-) x

int t = pop

sorted(a)

t=1

sorted(2):

if (-) ✓

push(t) →

return;

top sorted

2
3

push(t)

1
2
3

Question # 05

(DRY_RUN)

Suppose: our stack is

7	-
4	-
6	-

a) delete middle element:

if (—) ✗

int mid = size() / 2; mid = 3 / 2 = 1

del helper(0, mid);

① del helper(curr, mid)

curr = 0, mid = 1

if (—) ✗ 0 ≠ 1

int t = pop(); t = 7

del helper(curr + 1, mid);

② del helper(curr, mid)

curr = 1, mid = 1

if (—) ✓

pop()

4 (poped)

returns (to dh 1)

push(temp);

7
6

Question # 06

(DRY_RUN)

Suppose: our expression is { () }

is balanced

Stack S;

1) for (int i=0; i<str.length(); i++) ✓

ch = str[i];

ch = {

if (ch == {) ✓
{ s.push(ch); }
else if (—) ✗

{	}
---	---

2) for (—) ✓ 1 < 4

ch = str[i];

ch = (

if (ch == () ✓
{ s.push(ch); }
else if (—) ✗

(	)
{	}

3) for (—) ✓ 2 < 4

ch = str[i];

ch =)

if (—) ✗
else if (—) ✓
{

if (—) ✗

char topch = s.pop();

topch = (

if (—) ✗

Stack → { | }

for (—) ✓ 3 < 4

ch = str[i];

ch = }

if (—) ✗

else (—) ✓

{ if (—) ✗

char topch = s.pop();

topch = {

if (—) ✗

Question # 007

(DRY RUN)

Suppose

$n = 4$
arr = [1, 2, 3, 4]
ans = []

o) next greater element (int arr[], int n):

Stack S

int result[n];

o) for (int i = n-1; i >= 0; i--) ✓ $n >= 0$

while (!s.isempty() && s.peek() < arr[i])
if (s.isempty()) ✓

res[i] = -1;

else (—) ✗

s.push(arr[i])

→ arr[i] = 3

res[i] = -1

[3]

o) for (—) ✓ $i >= 0$

while (!s.isempty() && s.peek() < arr[i]) ✓

s.pop()

3 popped

if (s.isempty()) ✓

res[i] = -1;

else (—) ✗

s.push(arr[i]);

← arr[i] = 4

res[i] = {_, -1, -1}

[4, N]

o) for (—) ✓ $i >= 0$

while (—) {

if (—) {

else

res[i] = s.peek();

res[i] = {4, -1, -1}

s.push (x);

}

for (—) { if (—) {

Question # 08

(DRY_RUN)

Suppose stack is:

bool isPal()

{

if (—) {

Node* front = top;

(from Queue)

f == 0x1

return cph(top, front);

0x1	1	→ Top, front
0x2	2	
0x3	1	

bool cph(Node* curr, Node* F)

curr = 0x1, F = 0x1

if (—) {

bool res = cph(c->next, F);

②

bool cph(Node* c, Node* F)

c = 0x2, F = 0x1

if (—) {

bool res = cph(c->next, F);

③

bool cph(Node* c, Node* F)

c = 0x3, F = 0x1

if (—) {

bool res = cph(c->next, F);

bool cph(N* c, N* p)

if (—) ✓

return true;

// return to @

if (—) ✗

bool checkz (cur->d = f->d) 2:1 ✓

front = front->next; f = 0x0

return check;

// return to b

if (—) ✗ checkz true

bool checkz (c->d == f->d); c = 0x0, f = 0x0
2:2 ✓

front = front->next;

↓
continue

if's palindrome

Queue (All-Questions)

(Question #01)

class Queue

private;

int *ary;
int front;
int rear;
int cap;
int cnt;

// Data members
// first element
// last element
// tot capacity

public:

Queue(int s)

{

cap = s;
ary = new int[cap];
front = 0;
rear = -1;
cnt = 0;

}

// Constructor

// Initialization

~Queue()

{

delete [] ary;

}

// destructor

bool isep() { return cnt == cap; } // if no entry

bool isfull() { return cnt == cap; } // if full


```
void enqueue(int u)
```

```
{  
    if (isfull())  
    {  
        return;  
    }  
    rear = (rear + 1) % cap;  
    arr[rear] = u;  
    cnt++;  
}
```

// if cap = 10
0% suppose
rear = 0
arr[0] = 10

```
void dequeue()
```

```
{  
    if (isemp())  
    {  
        return;  
    }  
    front = (front + 1) % cap;  
    cnt--;  
}
```

// if nothing left

front = (front + 1) % cap;
cnt--;

1 % 4 = 1
c = 0

```
void display()
```

```
{  
    if (isemp())  
    {  
        return;  
    }  
    for (int i = 0; i < cnt; i++)  
    {  
        int ind = (front + i) % cap;  
        cout << arr[ind] << " ";  
    }  
}
```

let f = 0

int ind = (front + i) % cap;
cout << arr[ind];

0 % 2 = 0
ind = 0
then 1 % 2 = 1
then 2 % 2 = 0

Question # 02

Explained in basic operations of Queue
DRY_RUN

Suppose; User enters 5

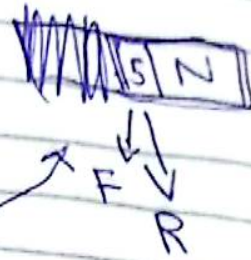
Node* nN = new Node(5);

if (—) ✓

F = R = NN;

return;

}



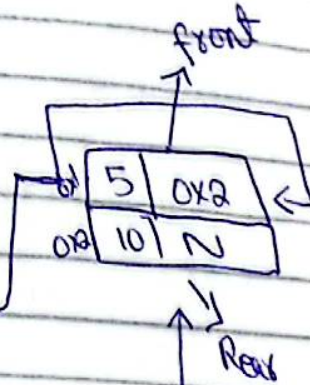
Now enter 10:

N* NN = new N(10);

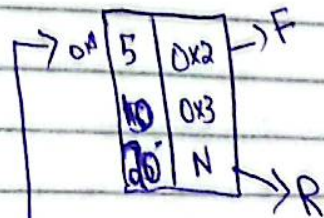
if (—) ✗

rear->next = NN;

rear = NN;



Now 20:



Now dequeue:

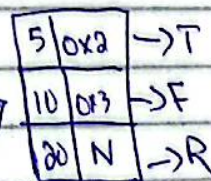
if (—) ✗

N* t = F;

F = F->Next;

if (F == Null) ✗

delete temp;



Question # 03

class PriorityQueue

{
private: arr;
int data;
int priority;
int cap;
int size;

// Data members

public:
PriorityQueue(int c)

// constructor

{
cap = c;
~~data~~ arr = new int[cap];
priority = new int[cap];
size = 0;

// init data

~PriorityQueue()

// destructor

{
delete [] arr;
delete [] priority;

void enqueue(int v, int p)

{
if (size == cap) { return; } // full

arr[size] = v;
priority[size] = p;
size++;

// insertions

// increment size

}

```

    }
    if (size == 0) { return; } // no element
    int hp = 0;

```

```

    for (int i = 1; i < size; i++)
    {
        if (priority[i] > priority[hp])
        {
            hp = i;
        }
    }

```

// finding element with highest priority
index found here

```

    for (int i = hp; i < size - 1; i++)
    {
        arr[i] = arr[i+1];
        priority[i] = priority[i+1];
    }
    size--;
}

```

// Shifting 1 step before to maintain linearity
// decrement size

```

void front()
{

```

```

    if (size == 0) { return; } // if no element

```

```

    int hp = 0;

```

```

    for (int i = 1; i < size; i++)
    {
        if (priority[i] > priority[hp])
        {
            hp = i;
        }
    }
}

```

// finding element with highest priority

Question # 04

// same codes for Stack & Queue
(Dry Run)

void reverse (Queue q, int k)

{ if (k <= 0 || k > q.size()) return;

Stack S;

1) for (int i = 0; i < k; i++)
{ q.enqueue(q.dequeue());

2) while (!S.empty())
{ S.push(q.dequeue());

3) int remaining = q.size() - k;

4) for (int i = 0; i < remaining; i++)
{ q.enqueue(q.dequeue());

k=2
Suppose

1	-
2	-
3	-
4	-

2	-
1	-

3	-
4	-
2	-
1	-



Question # 05

same code of Stack & Queue
(DRY - RUN)

isPalindrone(Queue & q)
if (q.empty())

Suppose



Stack S;

Queue T;

int qs = q.size(); $qs = 3$

for (int i = 0; i < qs; i++) ✓

int val = q.dequeue();

s.push(val);

T.enqueue(val);

q.enqueue(val);

}

val = 1

S → [1]

T → [1]



for (—) ✓

int v = q.de();

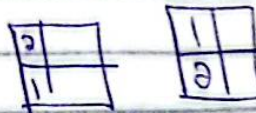
s.push(v);

T.enqueue(v);

q.enqueue(v);

}

val = 2



again pop() ✓ $0 < 3$

int $v = q.deg()$ $v = 1$

s.push(v);

1		
2		
1		

T.enqueue(v);

1		
2		
1		

q.enqueue(vall);

1		
2		
1		

1) bool ispan = true;

while (!T.isempty()) ✓

{
if (T.deg() != s.pop()) ✗
}

1) wh(—) ✓

{
if() ✗

1) wh() ✓

{
if(—) ✗

1) wh(—) ✗

return ispan;